

Haven

Sessionsrapport — 6 juli 2026

Från produktbeslut till live-bevisat budgetschema — på en dag

Den här sessionen stängde hela mottagar/budget/schema-komplexet: produktbeslutet om mottagar-lagring fattades och implementerades (#784), det pre-signerade budgetschemat kopplades in i betalvägen och bevisades live på Base Sepolia med noll signaturer över periodgränser (#769), dashboarden fick schema-signering och förnyelsebanner (#770), och mottagarna blev hanterbara i UI:t (#796). Dessutom: riktig webhook-alerting på delegat-monitorn (#777) och ett pilot-skriptindex som råkade rädda en borttappad fil (#778).

Mått	Värde
Stängda issues	6 (#777, #778, #784, #769, #770, #796)
Mergade PR:er	5 (#790, #792, #793, #794, #795, #797 — varav 2 migrationsgranskade)
Nya migrationer	2 (038 agent_recipients, 039 schemafönster)
On-chain-bevis	4 transaktioner på Base Sepolia (2 schema-enables + 2 cross-period-betalningar)
Testsviter, gröna	752/752 backend + 531/531 frontend
Ägarsignaturer vid periodgräns	0 — det var hela poängen

1. Problemet och idén

Sessionsrälsen (epic #733) gav Haven on-chain-policy per agent: mottagare, belopp per transaktion, kumulativ budget och giltighetstid bakas in i en Smart Sessions-session som kedjan själv upprätthåller. Men Smart Sessions budgetgräns är en **livstidsgräns** — den fylls aldrig på. Påfyllningen löstes i #734 med rotation: varje period får sin egen session, deterministiskt beräknad ur (agent, periodindex). Problemet: varje rotation krävde en **ägarsignatur**. Med dygnsbudget blir det en signatur om dagen — ohållbart för agenter som ska vara autonoma.

Nyckelinsikten bakom dagens bygge: eftersom rotationen är deterministisk är **framtiden beräknelig**. Session $N+30$:s exakta innehåll — och därmed dess identitet på kedjan — kan beräknas idag. Och Smart Sessions aktiverings-anrop tar en **lista**. Alltså kan EN ägarsignatur aktivera ett helt kvartal av sessioner, var och en tidslåst till sitt eget fönster:

FÖRE: rotation (#734) — en ägarsignatur VARJE periodgräns



EFTER: **pre-signerat schema (#769)** — EN signatur täcker hela fönstret



Varje session är **TIDSLÅST**: `validAfter = periodstart`, `validUntil = periodslut + en period nåd`.

En framtida session är DÖD före sitt fönster => max exponering = EN periods budget, precis som förut.

Vid periodgräns byter backend bara pekare mellan redan-signerade sessioner (lazy rollover)

— bokföring, inte auktoritet. Ingen människa, ingen cron, ingen ny signatur.

Figur 1 — Rotationsfriktionen och det pre-signerade schemat. Tidslåsningsen är det som gör konstruktionen säker: exponeringen ökar inte av att fler sessioner finns.

2. Dagens leveranser

Issue	Vad	PR
#777	Delegat-monitors fynd når nu en riktig kanal (Slack-webhook). Flankstyrd — pingar vid uppkomst och åter-uppkomst, tyst medan tillståndet består. Best-effort: ett misslyckat anrop rör aldrig skanningen.	#790
#778	Pilot/ops-skriptindex (README): DoD-flödet i ordning, räls-bevisen, ops-kadensen. Skrivandet avslöjade att create-dod-agent committats på en redan mergad gren och aldrig nått dev — filen räddades.	#792
#784	Produktbeslut A implementerat: agent_recipients-tabellen (migration 038) — en rad per (agent, token, mottagare), valfri label, valfri per-mottagare-budget där NULL ärver token-allowancen.	#793
#769	Schema-wiringen: migration 039 (fönsterkolumner), lazy rollover i authorize-vägen (flip bara efter lyckad prepare), pilot:schedule-live — och live-beviset med noll signaturer över två periodgränser.	#794
#770	Dashboard: schedule-API (state/build/confirm), budget-save bär schema-aktiveringen i samma signatur, förnyelsebanner vid ≤ 2 perioder kvar. Jargongfri copy assesterad i tester.	#795
#796	Mottagar-hantering: CRUD-API + kort på agentsidan (lista, lägg till, ta bort, budgetarv synligt) med "Sign to apply" — schemabyte i en signatur. Delad useApplySchedule-hook.	#797

3. Arkitekturen: från beslut A till rullande budget

Beslutsunkten (#784) var var agentens mottagare ska bo. Alternativ A — en egen tabell som speglar hur allowances modelleras — valdes. Det låser upp N mottagare per agent (en session per mottagare, eftersom Smart Sessions parameterregler AND:as ihop) och gav schemat det sista lagringsbenet det behövde:

Beslut A i schemat: mottagare + budgetarv + schemafönster (migration 038 + 039)



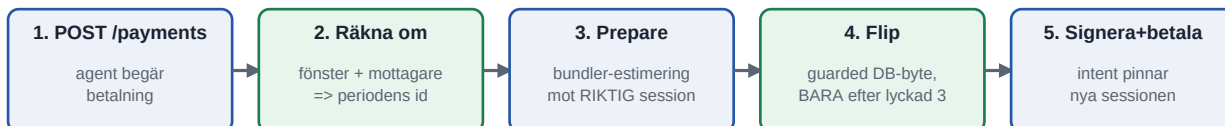
Den vassa kanten som avgjorde budgetsemantiken: en budget kan INTE aggregeras över mottagar-sessioner på kedjan — varje session bär sin egen kumulativa gräns.

Därför: per-mottagare-budget är sanningen (kedje-ärligt). Radens siffra gäller;
en off-chain-summay kan läggas ovanpå senare utan schemaändring.

Figur 2 — Datamodellen. Det enda tillstånd som lagras för schemat är FÖNSTRET (startperiod + antal); allt annat räknas om deterministiskt per betalning.

Att bara fönstret lagras är en designprincip, inte en slump: varje lagrad kopia av något som kan beräknas är en driftrisk. Authorize-vägen räknar om hela sessionsmatrisen ur databasens policy-indata (mottagare, budget efter arv, periodlängd, delegatnyckel) och jämför med kedjan — divergerar något är det stopp, inte gissning.

Lazy rollover — flippen sker bara efter att kedjan själv bevisat att sessionen är aktiverad



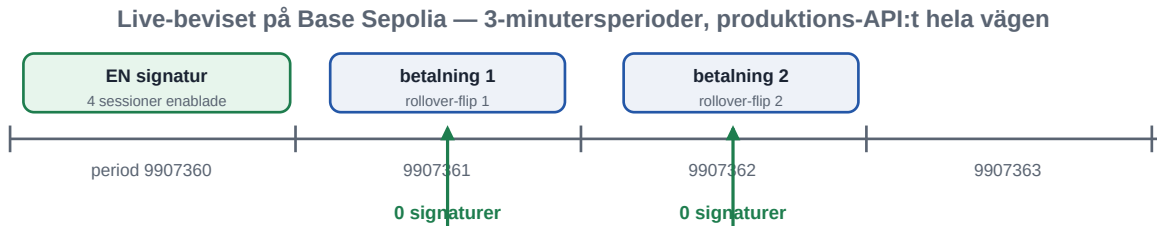
Fail-closed i VARJE söm: inget fönster / okänd mottagare / nollbudget / period utanför fönstret

=> exakt dagens beteende, ingen befintlig agent påverkas. Ett registrerat-men-aldrig-aktiverat fönster kan aldrig kapa pekaren: steg 3 misslyckas och databasen förblir orörd.

Figur 3 — Lazy rollover. Steg 3 är den eleganta biten: bundlerns gas-estimering kör den riktiga Smart Sessions-valideringen, så en lyckad prepare ÄR beviset på att sessionen finns på kedjan — ingen extra RPC, inget förtroende för klienten.

4. Live-beviset

Acceptanskriteriet var kompromisslöst: aktivera ett schema med kort testperiod, betala i period N, korsa gränsen **utan någon mänsklig handling**, betala igen med färsk budget. Det kördes mot dev-miljöns produktions-API med 3-minutersperioder:



Facit i databasen efteråt: session_permission_id = exakt period 9907362:s deterministiska id (0xb797...c4c) — pekaren hade flippat TVÅ gånger utan en enda mänsklig handling.

Färsk periodbudget vid båda betalningarna. Max exponering hela tiden: 0,10 USDC (en period).

Figur 4 — Cross-period-beviset. Första betalningen motionerade dessutom rollovern direkt: databasen pekade på period 9907360:s session när betalningen kom i 9907361.

Steg	Transaktion (Base Sepolia)
Schema-enable, EN signatur, 4 sessioner	0x4275...0ad37a5
Betalning period 9907361 (rollover-flip 1)	0x6d0a...c03dcf
Betalning period 9907362 (rollover-flip 2)	0xd01a...d25cfb
DB-facit: pekaren = periodens deterministiska id	0xb797...cd84c4c (verifierad via SELECT)

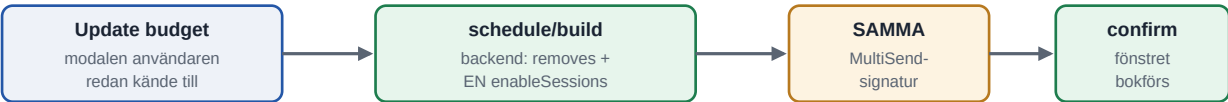
En operativ lärdom på köpet: första schemat (3 × 3 minuter) hann löpa ut medan databas-steget gjordes manuellt i Railways konsol. Omkörningen var billig (en signatur, 30 sekunder) — men momentet pekar på en verklig förbättring, se förslag 1 i avsnitt 8.

5. Dashboarden: signaturen användaren redan gör

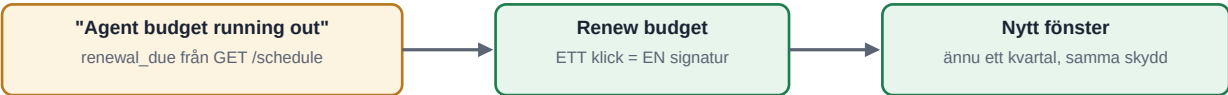
Designmålet från #770: **noll extra signaturer i happy path**. Schemat ska rida på signaturer användaren ändå gör — spara budget, förnya, ändra mottagare — aldrig kräva en egen ceremoni. Tre flöden byggdes:

Dashboard (#770 + #796): noll extra signaturer i happy path

1. Budget-save bär schemat



2. Förnyelsebanner vid <= 2 perioder kvar



3. Mottagarkortet (#796)

Lägg till/ta bort mottagare => raden sparas som bokföring => "Sign to apply" byter ut schemat i EN signatur.
Copy-regeln hålls av TESTER: orden "session"/"permission" får aldrig renderas — bara utfallsspråk ("budget", "recipients").

Figur 5 — Dashboardflödena. Backend returnerar råa inner-calls; frontend packar in dem i den MultiSend-batch den redan signerar. Backend signerar aldrig något.

Fail-open åt rätt håll: om schedule-API:t säger nej (inga mottagare, legacy-räls) går budget-saven igenom precis som idag — schemat är ett tillägg, aldrig en blockering. Och fail-closed åt andra hållet: en mottagarrad som sparats men inte signerats är ren bokföring — kortet säger "saved but not active yet", och kedjan vägrar betala tills ägarsignaturen landat.

6. Kodstrukturen — vad som ändrades

```
packages/backend/src/
  db/migrations/
    038_agent_recipients.ts          + mottagartabellen (beslut A)
    039_session_schedule_window.ts   + fönsterkolumner på agents
  lib/
    agent-recipients.ts              + accessor med budgetarv (EN plats)
    session-schedule-wiring.ts        + lazy rollover-beslutet + guarded flip
    delegate-balance-monitor.ts       ~ webhook-alerting (#777, flankstyrd)
  routes/
    agent-schedule.ts                 + GET state / POST build / POST confirm
    agent-recipients.ts               + CRUD (ägar-scopat, upsert, uint96-vakt)
    payments.ts                       ~ rollover i authorize-vägen
  scripts/
    schedule-live.ts                  + #769-livebeviset (pilot:schedule-live)
    db-schema-smoke.ts                ~ +2 money-path-frågor (8 totalt)

packages/frontend/src/
  hooks/useApplySchedule.ts           + delad build->sign->execute->confirm
  components/
    ScheduleRenewalBanner.tsx         + förnyelsebannern (~ refaktor till hook)
    AgentRecipientsCard.tsx           + mottagarkortet med "Sign to apply"
    EditAgentModal.tsx                 ~ budget-save batchar schema-actions
  lib/allowance-module.ts              ~ MultiSend-byggare för schema-txs

packages/qa-agent/src/pilot/
  README.md                           + skriptindex (#778)
  create-dod-agent.ts                  + RÄDDAD från tappad gren

+ = ny fil · ~ = ändrad. Totalt ~2 400 rader varav ~40 % tester.
```

7. Säkerhetsmodellen — vad som INTE ändrades

- **Max exponering är oförändrad.** En läckt sessionsnyckel kan som mest tömma innevarande periods budget (+ nådperiodens överlapp) — identiskt med manuell rotation. Framtida sessioner är döda före sitt validAfter.
- **Non-custody i varje led.** Haven konstruerar payloads och bokför pekare; ägaren signerar allt; kedjan är auktoriteten. CI-invarianterna (watch-only owner, inga nyckelsignerare, gas-only paymaster) gick gröna genom samtliga fem PR:er.
- **Fail-closed-sömmarna är uppräknliga och testade:** inget fönster, okänd mottagare, nollbudget, period utanför fönstret, misslyckad prepare — varje söm faller tillbaka till exakt dagens beteende.
- **Processgrindarna höll.** Båda migrationerna krävde och fick oberoende code-owner-granskning på GitHub (repo-regeln kunde inte kringgåas ens med admin-flagga — bra), och varje money-path-merge fick explicit ägar godkännande i sessionen.

8. Problem längs vägen — och vad de lärde oss

Problem	Lösning / lärdom
Första live-schemat (9 min fönster) hann löpa ut medan SQL kördes manuellt i Railway	Omkörning med 4 perioder. Lärdom: operatörssteg i databasen är flaskhalsen för tidskänsliga bevis — se förslag 1.
Railways webbkonsol är ett bash-skal, inte psql — SQL:en tolkades som kommandon	psql \$DATABASE_URL först, sedan klistra. Dokumenterat i minnet för framtida sessioner.

Problem	Lösning / lärdom
Två route-tester föll när schema-frågan dök upp mitt i flödet (positionella mockar)	Exakt scenariot #775-playbooken förutsåg. Nya tester mönster-matchar på SQL-fragment; de gamla lagades minimalt.
vi.useFakeTimers() frös Fastifys interna timers — testet time-out:ade	toFake: ['Date'] — mocka bara klockan, inte timers.
En lokal beforeEach(mockReset) fick varest att flagga en förhanterad rejection som ohanterad	Globala setup:ens clearAllMocks räcker — lägg inte lokala resets ovanpå. Tog tre hypoteser att isolera; probe-test i egen fil avgjorde.
create-dod-agent.ts fanns i git-historiken men inte på dev — committad på redan mergad gren	Upptäcktes av att #778-indexet skrevs. Räddad med git checkout från den tappade committen. Dokumentationsarbete som regressionstest.
gh pr create med backticks i kroppen lät zsh command-substituera mitt i PR-texten	PR-kroppar skrivs alltid via --body-file, aldrig inline med skal-metatecken.

9. Förslag till förbättringar

Prioriterat: först det som tar bort manuella moment ur redan byggda flöden, sedan robusthet, sist förädling.

1. Ta bort SQL-momentet ur pilot-flödena (störst friktion idag)

Både DoD-flödet och schema-beviset kräver att en operatör kör SQL i Railway-konsolen — det raceade bokstavligen mot periodklockan idag. Nu när schedule/confirm och recipient-CRUD finns som API borde pilot-skripten anropa dem i stället för att skriva ut SQL-block. Det som saknas är en dev-inloggning i pilot-env och att skriptet postar mot API:t. Uppskattad insats: liten. Vinst: livebevis utan människa i loopen.

2. Exponera betalningens session i API-svaret

Det pinnade session_permission_id:t syns inte i GET /payments/:id — dagens verifiering krävde en manuell SELECT. Fältet finns redan i intents-tabellen; att exponera det (för ägaren, inte agenten) gör rollovern observerbar och felsökbar utan databastillgång.

3. Multi-sig-flödet för schema-apply saknas

Banner och mottagarkort vaktar bort trösklar >1 med ett felmeddelande, och confirm körs aldrig för föreslagna transaktioner — fönstret bokförs alltså inte när en multi-sig-Safe till slut exekverar. Rätt lösning: häng confirm på exekverings-notifieringen (samma mönster som owner-hanteringens propose-flöde), eller låt backend upptäcka fönstret on-chain i efterhand (se förslag 4).

4. Confirm kunde verifiera mot kedjan i stället för att lita på klienten

schedule/confirm bokför det klienten påstår. Det kan inte flytta pengar (rollovern prepare-verifierar), men ett felbokfört fönster ger förvirrande UI-läge. Backend kan verifiera med isSessionEnabled för första periodens id innan den skriver — en RPC, eliminerar hela klassen. Löser även multi-sig-luckan i förslag 3 om den körs som svep.

5. Budgetändring utan apply kan strypa betalningar — gör diffen synlig

Rollovern räknar om sessionerna ur databasens nuvarande policy. Ändras en mottagares budget via API utan att schemat byts ut divergerar beräkningen från kedjan: prepare misslyckas och betalningarna stannar (fail-closed — rätt riktning, men tyst). Dashboarden täcker sig med pending-apply-flaggan; API-vägen borde också svara med en varning, och GET /schedule kunde exponera "drift detected" genom att jämföra beräknat id mot kedjan.

6. Notifiering när förnyelsen närmar sig

Bannern är pull-baserad — ägaren måste besöka sidan. renewal_due-flaggan finns redan; koppla den till mejl/push (samma kanal som approval-notiserna) så att en agent aldrig stannar för att ingen råkade titta. Naturlig cron: samma timvisa jobb som delegat-monitorn.

7. Multi-token-agenter i mottagarkortet

Kortet binder nya mottagare till agentens FÖRSTA konfigurerade token. Korrekt för pilotens enkel-token-fall, men en agent med USDC + EURE behöver en token-väljare per rad. Backend stödjer det redan — bara UI-arbete.

8. Live-bevisa N×M-matrisen

Allt är byggt för N mottagare × M perioder, men live-beviset kördes med en mottagare. Kör pilot:schedule-live med två mottagarrader (bygget stödjer det — en kombinerad enableSessions) och betala till båda i samma period: då är även matris-fallet bevisat, inte bara designat.

Avslutning

Förra rapporten slutade med att rotationsfriktionen var en känd svaghet med en designad lösning. Den här sessionen tog designen hela vägen: beslut, schema, wiring, on-chain-bevis, dashboard och hantering — med säkerhetsmodellen intakt i varje steg och processgrindarna (migrationsgranskning, money-path-godkännanden) respekterade hela vägen. Kvar på session-rälsens karta är mainnet-beslutet, Stage 3 (legacy-pensionering och 7710-spåret) och förädlingarna ovan — men kärnlöftet är nu infriat och bevisat: **en signatur, ett kvartal, en agent som betalar själv inom ramar kedjan upprätthåller.**

Genererad 2026-07-06 · Haven-AI · dev-gren efter merge av PR #797